```
    [99] => 1
    [100] => 1
)
```

The default value for the mode argument is 0; if we explicitly pass 1, then the function only returns non-zero counts, as we see above. If we passed 2, it would return only zero-counted byte values. The final two modes will return strings containing only the bytes found or only the bytes not found:

```
var_dump(count_chars("hello world", 3));
var_dump(count_chars("abcd", 4));
```

```
Output:
string(8) " dehlorw"
string(252) "
```

```
!"#$%&'()*+,-./01234
```

Mode 3 returns each byte encountered (in order), and mode 4 gives us all of the bytes not encountered. This includes all of the non-printing control bytes in the *0x00–0x1F* range, which is why the output string looks that way: the first 32 bytes are control characters (including the "bell" character *0x07* – your terminal might make a noise when running the second line of the example), and the first display characters are the space at *0x20* and the exclamation mark at *0x21*.

`strlen()`, `mb_strlen()` + secret bonus

The simplest case is usually "How long is my string?" In other words, how many bytes are in this buffer? This is often a useful thing to know, perhaps for iterating over strings or for measuring how many bytes to tell a remote client to expect from your program (e.g., in a `Content-Length:` header). Good old `strlen()` has this use case covered.

```
$response = '<!DOCTYPE html><html><head></head><body>Lots of lovely
content.</body></html>';
```

```
$contentLength = strlen($response);
var_dump($contentLength);
var_dump(strlen("abc\x00def"));
```